Code Implementation of Forward Kinematics and Other Methods

Summary:

This lesson turns the forward kinematics from Lessons 3 and 4 into working Python code. On Day 1, students write two short functions, `dh_matrix` and `forward_kinematics`, that turn the arm's DH table into a tip position, and check them against the hand-worked answers from Lesson 4 with a pass/fail test file. On Day 2, students compare the DH code with Lesson 3's trigonometry, meet a second method for forward kinematics (screw theory, also called the product of exponentials), and use their function to sweep through many poses and map where the arm's tip can reach. By the end of the lesson, every student has a tested `forward_kinematics` function that the inverse kinematics lessons (6–8) can build on.

Objective:

By the end of this lesson, students will be able to:

- Translate the arm's DH table into Python code, one table row for each call of `dh_matrix`
- Write a `forward_kinematics` function that multiplies the three joint matrices with NumPy and returns the tip position (x, y, z)
- Test the function against hand-calculated answers with a pass/fail check, and name the two most common bugs (degrees vs. radians, and the order of multiplication)
- Describe a second method for forward kinematics (screw theory), run it, and confirm that it agrees with the DH result
- Use the function to sweep through many joint angles and describe the region the arm's tip can reach

Standards:

NGSS

- Engineering Design: HS-ETS1-2 (break a complex problem into smaller problems), HS-ETS1-3 (evaluate solutions against trade-offs), HS-ETS1-4 (use a computer model to test a solution)
- Science & Engineering Practices: Developing and Using Models; Using Mathematics and Computational Thinking; Analyzing and Interpreting Data
- Crosscutting Concepts: Systems and System Models; Patterns

Vocabulary:

Function: A named block of code that takes inputs and gives back an output. In this lesson, `dh_matrix` and `forward_kinematics` are functions.

NumPy: A Python library for working with arrays and matrices. It is imported as `np`, and a 4 × 4 matrix is stored as a 4 × 4 NumPy array.

Radians: A unit for angles in which 180° = π (about 3.14159). NumPy's `sin` and `cos` expect radians, so `np.radians` converts degrees into radians first.

Matrix multiplication (@): The symbol `@` multiplies two matrices. The symbol `*` does not; it multiplies entry by entry.

Slicing: Picking out part of an array. `T[:3, 3]` means the top three entries of the last column of T, which is the tip position.

Test: A short piece of code that runs a function on poses with known answers and reports PASS or FAIL for each one.

np.allclose: A NumPy check that two numbers or lists are equal to within a small allowed difference. It is used instead of == because computed answers carry tiny rounding errors.

Zero pose: The arm standing straight up with all three joint angles at 0°. The tip is at (0, 0, 500).

Screw theory (product of exponentials): A second method for forward kinematics. It starts from the arm in its zero pose and describes each joint by its turning axis.

Screw axis: The line a joint turns around, described by its direction and one point on the line.

Reachable region: The set of all points the arm's tip can reach.

Motor tick: One count of a servo's position. The servos used here count 4096 ticks per full turn.

Resources & Materials:

- Lesson 5 slideshow, Days 1 and 2 (37 slides; the slide numbers in the Procedures refer to it)
- Computers with Python 3 and NumPy (one per pair), plus matplotlib for the Day 2 reach plot
- Student files: `fk_starter.py` (saved as `fk.py`), `check_fk.py`, `screw_helper.py`, `fk_screw_starter.py` and `workspace_sweep.py`
- Teacher solutions: `fk.py`, `fk_screw.py` and `compare_methods.py` (the same code is shown on the slides, so hold the files back until students have tried it themselves)
- Calculators for checking Try It Yourself #3 by trigonometry
- The assembled 4-DOF arm from Lesson 2, to show the zero pose (arm straight up) and the three turning axes on Day 2 (optional)
- The arm powered and connected to a computer, to read motor ticks for the last Day 2 slide (optional)
- All code uses link lengths L1 = L2 = 250 mm, as in Lessons 3 and 4. If the built arm measures differently, change the two constants at the top of `fk.py` and the expected answers in `check_fk.py`.

Procedures:

Day 1: Forward Kinematics in Code

- **Opening (4 min)** — Display the title slide, then "Where Lesson 4 Left Off" and "From Hand Steps to Code Steps" (slides 1–3). Recall that Lesson 4 found (301.78, 301.78, 176.78) for θB = θS = θE

= 45° by building and multiplying three matrices by hand. Today every hand step becomes one line of Python: the DH table becomes three `dh_matrix` calls, the multiplication becomes `@`, and the last column becomes `T[:3, 3]`.

- **Setting up Python and NumPy (6 min)** — Show "Setting Up Python" (slide 4): students save `fk_starter.py` as `fk.py` and check that `import numpy as np` runs. Then show "Four NumPy Tools" (slide 5): `np.radians` (NumPy's sin and cos expect radians, not degrees), `np.array`, `@` for matrix multiplication, and slicing such as `A[:2, 2]`.
- **Try It Yourself #1 (4 min)** — Students predict, then run `np.round(np.sin(np.radians(30)), 2)`, `np.round(250 * np.cos(np.radians(45)), 2)` and `np.array([[1, 2], [3, 4]]) @ np.array([1, 1])` (slide 6); reveal 0.5, 176.78 and [3 7]. Point out that 176.78 is the same number as ΔX1 in Lesson 3 and the rotation example in Lesson 4.
- **Writing dh_matrix (10 min)** — Show "Step 1: A Function for One Joint" (slide 7): four numbers go in (θ, d, a, α) and one 4×4 matrix comes out. Students type the matrix from Lesson 4, slide 22, into the function, starting with the two `np.radians` lines ("Writing dh_matrix", slide 8). Then test one joint ("Test One Joint", slide 9): `dh_matrix(45, 0, 250, 0)` and `dh_matrix(-45, 0, 250, 0)` should have last columns (176.78, 176.78, 0) and (176.78, −176.78, 0), the same as A_2 and A_3 in Lesson 4.
- **Writing forward_kinematics (12 min)** — Show "Step 2: The DH Table in Code" (slide 10): each row of the DH table becomes one `dh_matrix` call, with `90 - theta_s` and `-theta_e` in the θ column as in Lesson 4, and the link lengths stored once as `L1` and `L2`. Show "Step 3: Multiply and Read the Tip" (slide 11): `T = A1 @ A2 @ A3` in joint order, then the last column of T is the tip. Students write the function, checking against "The Whole Function" (slide 12) only when stuck.
- **Run the worked example (3 min)** — Show "Run the Worked Example" (slide 13): `show(forward_kinematics(45, 45, 45))` prints `[301.78, 301.78, 176.78]`, the same answer as Lesson 4. Explain that `show` only rounds to 2 decimals so the output is easy to read.
- **Try It Yourself #2 (5 min)** — Students run Lesson 4's Try It Yourself #4 and #5 in code (slide 14): θB = 30°, θS = 60°, θE = 0° and θB = 120°, θS = 0°, θE = 90°. Reveal [375.0, 216.51, 250.0] and [−125.0, 216.51, 250.0]; Python drops trailing zeros, so 375.00 prints as 375.0. Students compare with their hand answers from Lesson 4.
- **A pass/fail test (8 min)** — Ask how we know the function is right for every pose, not just one ("Why Write a Test?", slide 15). Show the seven test poses, including easy ones that can be checked by sight: arm straight up gives (0, 0, 500) and arm straight out along x gives (500, 0, 0). Walk through the loop in `check_fk.py` ("The Test File", slide 16), including why it compares with `np.allclose` instead of `==` (rounding). Students run it ("Run the Test", slide 17) and should see `7 of 7 tests passed`.
- **Bug hunt (6 min)** — Show three broken versions and their wrong answers for 45°, 45°, 45° ("Bug Hunt", slide 18): forgetting `np.radians` gives [281.43, 274.4, 190.18], multiplying in the wrong order gives [426.78, −176.78, 0.0], and forgetting the θ conversions gives [125.0, 125.0, 426.78]. Pairs match each bug to its symptom, then reveal "Bug Hunt: Answers" (slide 19). Teaching point: the straight-up test, (0, 0, 0) should give (0, 0, 500), catches all three.
- **Day 1 recap (2 min)** — Show "Day 1 Recap" (slide 20): one function per joint, one function for the arm, one test file to trust them. Preview Day 2: the same arm can be described in other ways, and the function can map where the tip can reach.

Day 2: Other Methods and the Arm's Reach

- **Recap (3 min)** — Show "Day 2: Where We Left Off" (slide 21) and review Day 1's `forward_kinematics`. Quick-fire questions: What does `dh_matrix` return? (A 4×4 matrix.) Why `@` and not `*`? (`*` multiplies entry by entry.) What should the straight-up pose (0, 0, 0) give? ((0, 0, 500).)
- **Lesson 3 in code (6 min)** — Show "Lesson 3 in Code" (slide 22): the triangles of Lesson 3 as a short function, `trig_fk`, which students type into a new file, `compare_methods.py`, next to `fk.py` (the file starts with `import numpy as np` and `from fk import forward_kinematics as fk, L1, L2`). The function uses r = L1·sin θS + L2·sin(θS + θE), z = L1·cos θS + L2·cos(θS + θE), x = r·cos θB and y = r·sin θB. Then "Do the Methods Agree?" (slide 23): the loop, added below `trig_fk` in the same file, tries 2,000 random poses and prints `True` when the biggest difference between `forward_kinematics` and `trig_fk` is below 0.000001 mm.
- **Try It Yourself #3 (6 min)** — Students find r and z by trigonometry with a calculator, then check with code (slide 24): θB = 90°, θS = 90°, θE = 45° and θB = 0°, θS = 30°, θE = 60°. Reveal [0.0, 426.78, −176.78] and [375.0, 0.0, 216.51]. Discuss why z is negative in the first one (θS + θE = 135°, so the forearm points below horizontal).
- **A second way to describe the arm (10 min)** — Show "A Second Way to Describe the Arm" (slide 25): DH needs a frame on every joint and a table, whereas screw theory (also called the product of exponentials) starts from the arm in its zero pose and describes each joint by its turning axis. Show "The Zero Pose and the Axes" (slide 26): arm straight up, tip at (0, 0, 500); base axis points along z through the origin, shoulder axis along y through the origin, elbow axis along y through (0, 0, 250). If the arm is available, point to each axis. Show "The Screw Recipe" (slide 27): turn the elbow first, then the shoulder, then the base, applied to the zero-pose tip. This is the reverse of the DH reading order but gives the same answer. Students do not derive the formulas; `screw_helper.py` supplies `screw_axis` and `screw_matrix`.
- **Screw theory in code (8 min)** — Show "Screw Theory in Code" (slide 28): the zero-pose matrix M and the three axes S1, S2 and S3, each made from a direction and one point on the axis. Walk through "Worked Example: One Turn at a Time" (slide 29) for 45°, 45°, 45°: the elbow turn takes the tip (0, 0, 500) to (176.78, 0, 426.78), the shoulder turn to (426.78, 0, 176.78) and the base turn to (301.78, 301.78, 176.78), the same answer as DH.
- **Try It Yourself #4 (7 min)** — Students save `fk_screw_starter.py` as `fk_screw.py` and write the one-line body of `forward_kinematics_screw` (slide 30). Then they change the import line in `check_fk.py` to test it. Reveal the solution, `T = screw_matrix(S1, theta_b) @ screw_matrix(S2, theta_s) @ screw_matrix(S3, theta_e) @ M`, and `7 of 7 tests passed`.
- **Three methods compared (3 min)** — Show "Three Methods Compared" (slide 31): trigonometry needs new triangles for every new arm, DH needs a table and one recipe, and screw theory needs the zero pose and the axes. Ask: if the arm had six joints, which would you choose, and why?
- **Where can the tip go? (8 min)** — Show "Where Can the Tip Go?" (slide 32): the function can be called thousands of times, so we can try many poses and see where the tip lands. Walk through `workspace_sweep.py` ("Sweeping the Joint Angles", slide 33): θS and θE from 0° to 90° in 5° steps with θB = 0°, which prints 361 poses, farthest 500.0 mm and nearest 353.55 mm. Show the

plot ("The Reachable Region", slide 34) and explain that turning the base spins this picture around the z axis. These ranges are practice values, not the real arm's limits.

- **Try It Yourself #5 (4 min)** — Students change the elbow range to 0°–150° and predict the new nearest distance before running (slide 35). Reveal 589 poses, farthest 500.0 mm and nearest 129.41 mm, which is 2 × 250 × cos 75°: a bent elbow lets the tip come much closer to the shoulder.
- **Optional: from motor ticks to angles (3 min)** — If time allows, show "Optional: From Motor Ticks to Angles" (slide 36): the servos count 4096 ticks per turn, so 1 degree is 4096 ÷ 360 ticks, and the base motor is geared 3 to 1. With the zero-pose ticks as the reference, 512 ticks is 45° on the shoulder or elbow and 1536 ticks is 45° on the base. The zero-pose tick values must be measured on the built arm (placeholder `ZERO_TICKS` in the code), and the motor direction may need a sign flip.
- **Wrap-up (2 min)** — Show "What We Learned" (slide 37): a tested function turns three joint angles into a tip position, three methods give the same answer, and the function can map the arm's reach. Preview Lesson 6: inverse kinematics goes the other way, from a target position back to the joint angles.

Assessment:

Assess with working code. On Day 1, each student pair shows a `forward_kinematics` function that prints `7 of 7 tests passed` in `check_fk.py`. On Day 2, pairs show `forward_kinematics_screw` passing the same test, and complete the Try It Yourself problems in the slideshow (#1–#5). Students check each answer against the solution on the slide, and can check #2 to #5 by running their own code.

Use these questions as an exit ticket or short quiz. The answers are in red.

- Why must the code call `np.radians` before `np.sin` or `np.cos`?
  Answer: NumPy's `sin` and `cos` expect radians. If they are given degrees, NumPy reads 45 as 45 radians and the answer is wrong ([281.43, 274.4, 190.18] for 45°, 45°, 45°).

- What does `dh_matrix(theta, d, a, alpha)` return, and what do the four inputs mean?
  Answer: A 4 × 4 NumPy array, the matrix for one joint. θ is a turn about z (degrees), d is a slide along z (mm), a is a slide along x (mm) and α is a turn about x (degrees).

- After `T = A1 @ A2 @ A3`, which expression gives the tip position (x, y, z)?
  Answer: `T[:3, 3]`, the top three entries of the last column of T.

- Why does the code use `@` and not `*` to multiply two matrices?
  Answer: `@` is matrix multiplication. `*` multiplies entry by entry, which gives a different matrix.

- What goes wrong if the matrices are multiplied as `A3 @ A2 @ A1`?
  Answer: The matrices must follow the order of the joints. In the wrong order the function returns the wrong point: [426.78, −176.78, 0.0] instead of [301.78, 301.78, 176.78] for 45°, 45°, 45°.

- What should `forward_kinematics(0, 0, 0)` give, and why is it a good test pose?
  Answer: (0, 0, 500). The arm stands straight up, so the tip is L1 + L2 = 250 + 250 = 500 mm above the base. The answer is easy to check by sight, and it catches all three bugs from the Bug Hunt.

- Why does `check_fk.py` compare answers with `np.allclose` instead of ==?
  Answer: The expected answers are rounded to 2 decimals, but the function returns longer decimals (such as 301.7767). `np.allclose` accepts a difference of up to 0.01, whereas == would fail.

- Name the three methods for forward kinematics from Lessons 3 to 5, and say what each one needs.
  Answer: Trigonometry needs new triangles for every new arm. DH needs a frame on each joint, a table and one recipe. Screw theory needs the arm's zero pose and each joint's turning axis.

- The sweep tries θS and θE from 0° to 90° in 5° steps with θB = 0°. How many poses is that, and why is the farthest distance exactly 500.0 mm?
  Answer: 19 × 19 = 361 poses. The farthest tip is at θE = 0°, when the forearm continues straight on from the upper arm: 250 + 250 = 500 mm.